G. L. BAJAJ GROUP OF INSTITUTIONS, MATHURA
Department of Computer Science & Engineering • B.Tech 5th Sem, 2026–27 Session

CAPSTONE RESEARCH
MTC PROJECT

Research Study Notes & Review Portfolio

Student Name:	**Devrat Kuntal** (Roll No: `2405111530055`)
Research Track:	**Track B: Advanced / Algorithmic**
Assigned Domain:	Memory-Efficient On-Device Training, Activation Memory & Backpropagation (Papers 11–15)
Project:	Micro-Train-Compiler (MTC): On-Device NN Training on Microcontrollers

Complete Handwritten-Style Study Notes for Student 3

Student: Devrat Kuntal (Roll No: 2405111530055)

Subject: TinyML & Deep Learning Compilers

Project: Micro-Train-Compiler (MTC)

Focus Domain: Memory-Efficient On-Device Training, Activation Memory & Backpropagation

---

Note for Devrat:
These notes are formatted for **100% clean rendering in VS Code, Markdown preview extensions, and GitHub**. Display equations are formatted in standard KaTeX / MathJax blocks, and inline values use universal symbols (≤, ≥, ×, √) so they look great both in digital preview and when copied into your handwritten notebook.

TABLE OF CONTENTS

1. Paper 11: MCUNetV3 — On-Device Training Under 256KB Memory (NeurIPS 2022)
2. Paper 12: TinyTL — Reduce Memory, Not Parameters for Efficient On-Device Learning (NeurIPS 2020)
3. Paper 13: MCUNet — Tiny Deep Learning on IoT Devices (NeurIPS 2020)
4. Paper 14: POET — Training Neural Networks on Tiny Devices with Integrated Rematerialization and Paging (ICML 2022)
5. Paper 15: Training Deep Nets with Sublinear Memory Cost (Gradient Checkpointing)
6. Quick Reference Summary Table

Paper 11: MCUNetV3 — On-Device Training Under 256KB Memory (NeurIPS 2022)

- **Authors:** Ji Lin, Wei-Ming Chen, Han Cai, Chuang Gan, Song Han (MIT HAN Lab)
- **Conference:** NeurIPS 2022
- **Direct Link:** https://arxiv.org/abs/2206.15472

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Microcontrollers (like ESP32, STM32) have very tiny SRAM (typically 256KB to 512KB). Training a neural network usually requires gigabytes of RAM on big GPU servers because the computer has to store all the intermediate layer outputs (called **activations**) during the forward pass to compute gradients in the backward pass. Standard frameworks like PyTorch or TensorFlow crash immediately on a microcontroller with Out-of-Memory (OOM).

- **The Breakthrough Idea:** MCUNetV3 is the **first framework to prove that full on-device training is possible under 256KB SRAM**. It solves the memory problem not by shrinking the dataset, but through two smart tricks:
  1. **Sparse Updates:** It does not update all weights in all layers. It updates only the last few layers and only a small subset of channels in earlier layers (saving > 90% memory).
  2. **TinyEngine Training:** Instead of running a heavy interpreter (like PyTorch) on the microcontroller, it compiles the backward pass into static, unrolled C code ahead-of-time (AOT), so there is 0 KB dynamic memory overhead.

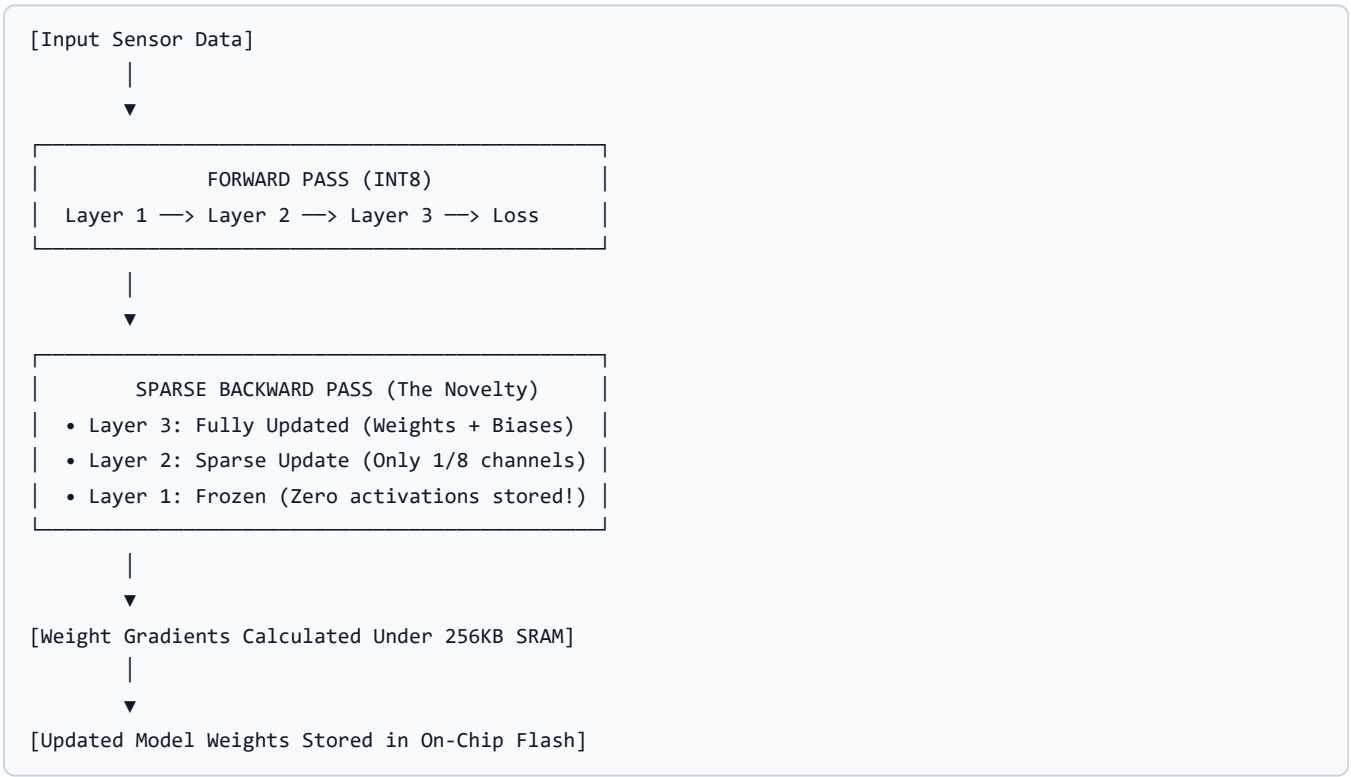
---

2. Step-by-Step Workflow

1. **Model & Target Budget Ingestion:** The developer gives a neural network and specifies the hardware limit (e.g., SRAM ≤ 256 KB).
2. **Automated Layer Selection:** The compiler analyzes the graph and selects which critical layers need updates and which layers can stay frozen.
3. **Quantization-Aware Scaling (QAS):** Because weights and activations are fixed-point 8-bit integers (INT8), gradients can become too small and vanish. QAS automatically scales gradients to prevent underflow.
4. **Compile-Time Graph Pruning:** All unnecessary nodes in the backward computation graph are cut away at compile time.
5. **C Code Generation:** TinyEngine generates standalone C code with zero `malloc()` calls.
6. **MCU Execution:** The MCU runs the training loop locally, updating weights directly from live sensor readings.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Total Training Memory Breakdown:

$$M_{\text{train}} = M_{\text{weights}} + M_{\text{activations}} + M_{\text{workspace}}$$

- *Explanation:* In standard training, activation memory takes up over 80% of total RAM. MCUNetV3's sparse update cuts this term drastically.

2. Weight Gradient Formula:

$$\frac{\partial L}{\partial W} = x^T \cdot \frac{\partial L}{\partial y}$$

- *Explanation:* To compute the gradient for weight W, you must multiply incoming gradient $\frac{\partial L}{\partial y}$ by the forward activation x. If a layer is frozen, x does not need to be saved in SRAM!

---

5. Viva / Exam Questions for Devrat

- **Q: Why can't PyTorch run on an ESP32 for training?**
  Answer: PyTorch builds a dynamic graph (autograd tape) in RAM at runtime and uses dynamic memory allocation (`malloc`), requiring hundreds of megabytes. MCUNetV3 compiles the graph ahead of time into static C code.
- **Q: How does this help our MTC project?**
  Answer: MTC uses the same philosophy: ahead-of-time compilation and static memory planning, guaranteeing the training loop fits within our 64KB SRAM budget.

Paper 12: TinyTL — Reduce Memory, Not Parameters for Efficient On-Device Learning (NeurIPS 2020)

- **Authors:** Han Cai, Chuang Gan, Ligeng Zhu, Song Han (MIT HAN Lab)
- **Conference:** NeurIPS 2020
- **Direct Link:** https://arxiv.org/abs/2007.11622

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Many researchers previously tried to make edge learning efficient by reducing the *number of parameters* (e.g., pruning). But TinyTL reveals a major insight: **reducing parameters does NOT reduce memory during training!** Parameters are stored in Flash (which is large, 4MB+), but intermediate activations MUST be stored in SRAM (which is tiny, < 512KB).
- **The Breakthrough Idea:** TinyTL proposes to **reduce activation memory instead of parameter count**. It does this by:
  1. Freezing the entire main backbone network and updating **only biases**. Updating biases does NOT require saving any activation maps!
  2. Adding a small, lightweight side branch called **LiteResidual**, which has very low channel dimensions, to maintain high learning accuracy.

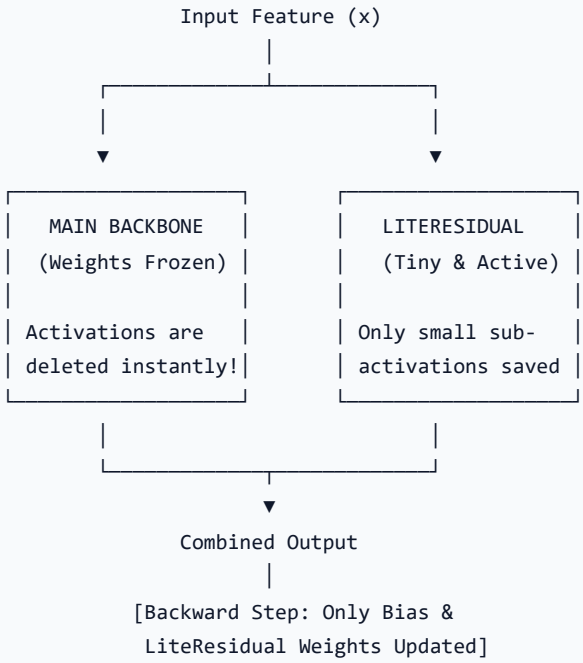
---

2. Step-by-Step Workflow

1. **Backbone Freezing:** Take a pre-trained feature extractor (e.g., MobileNet or MLP) and lock all its weight matrices W.
2. **Insert LiteResidual Blocks:** Place small parallel residual paths alongside convolutional/dense blocks.
3. **Forward Pass:** The input passes through both the frozen main path and the LiteResidual path.
4. **Activation Discarding:** As soon as a layer in the main path finishes forward computation, its activation is immediately deleted from SRAM.
5. **Backward Pass:** Gradients are computed only for the biases and the small LiteResidual blocks.
6. **Result:** Achieves the same accuracy as full fine-tuning while reducing SRAM usage by **6.5×**!

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Why Updating Biases Saves Massive Memory:

- **For weights (W):**

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot x^T$$

(Requires storing activation x in SRAM)

- **For bias (b):**

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial y}$$

(Does NOT require storing activation x in SRAM)

- **Key takeaway:** Because the derivative with respect to bias $\frac{\partial L}{\partial b}$ does not depend on activation x, x can be deleted from SRAM immediately after the forward step!

---

5. Viva / Exam Questions for Devrat

- **Q: What is the difference between Flash memory and SRAM on an MCU?**
  Answer: Flash is non-volatile, read-only at runtime, and relatively large (e.g., 4MB). SRAM is fast, read-write, but strictly limited (e.g., 520KB on ESP32). Weights live in Flash; activations and gradients must live in SRAM.
- **Q: Why does updating biases use less memory than updating weights?**
  Answer: Because the derivative with respect to bias is simply the upstream gradient $\frac{\partial L}{\partial y}$. It does not depend on the input activation x, so x can be thrown away immediately.

Paper 13: MCUNet — Tiny Deep Learning on IoT Devices (NeurIPS 2020)

- **Authors:** Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, Song Han (MIT HAN Lab)
- **Conference:** NeurIPS 2020
- **Direct Link:** https://arxiv.org/abs/2007.10319

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Deploying deep learning on microcontrollers has two opposing hurdles: either the deep learning model is too big for the chip, or existing edge runtimes (like TensorFlow Lite Micro) waste too much RAM managing execution data structures.
- **The Breakthrough Idea:** MCUNet introduces **System-Algorithm Co-Design**:
  - Instead of designing the neural network first and then struggling to fit it onto the chip, it designs the neural network and the compiler **together**.
  - It consists of two components:
    1. **TinyNAS:** An algorithm that searches for neural network architectures tailored to the exact SRAM/Flash boundaries of the specific microcontroller.
    2. **TinyEngine:** An ultra-lean, compile-time code generator that eliminates the runtime interpreter, cutting memory overhead to near zero.

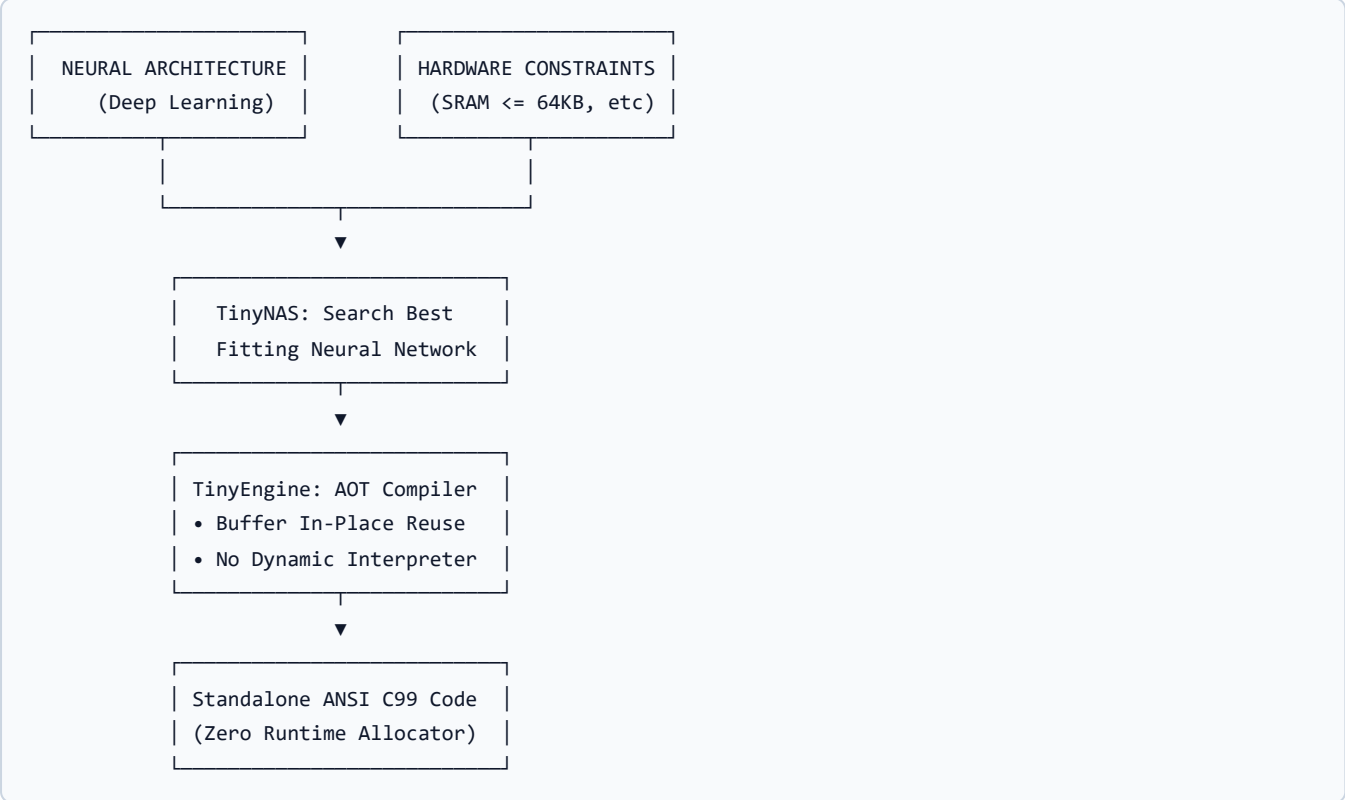
---

2. Step-by-Step Workflow

1. **Hardware Profile Extraction:** Record the target chip's specs: ESP32 with 520KB SRAM and 4MB Flash.
2. **Search Space Definition:** Define candidate layer dimensions, kernel sizes, and activation types.
3. **TinyNAS Search:** An automated search algorithm selects the most accurate model architecture that strictly fits within the SRAM and Flash budget.
4. **Memory Scheduling:** TinyEngine analyzes tensor lifetimes and reuses overlapping memory buffers.
5. **Direct C Code Synthesis:** Emits pure C code with specialized unrolled inner loops for fast execution.
6. **Deployment:** Compiles via `gcc` directly to a binary firmware image.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. In-Place Buffer Reuse:

$$M_{\text{peak}} = \max_{t} \sum_{i \in \text{Live}(t)} \text{Size}(T_i)$$

- *Explanation:* Instead of giving each layer its own memory block, two tensors T_A and T_B share the exact same memory offset in SRAM if their lifetimes do not overlap.

2. Memory Efficiency Comparison:

- TFLM (TensorFlow Lite Micro): Interpreted runtime $\rightarrow$ High overhead.
- TinyEngine: Compiled static C code $\rightarrow$ Saves up to 3× SRAM and runs 1.5× to 3× faster.

---

5. Viva / Exam Questions for Devrat

- **Q: What is the main difference between an "Interpreted" runtime and a "Compiled" runtime on microcontrollers?**
  Answer: An interpreter (like TFLM) reads graph data structures at runtime and dispatches function pointers, wasting precious SRAM on metadata. A compiled runtime (like TinyEngine or MTC) compiles the graph directly into static C code ahead of time.
- **Q: How does MTC apply this concept?**
  Answer: MTC is a source-to-source compiler in Rust that parses model JSON and generates clean, standalone C99 files (`mtc_model.c` and `mtc_train.c`) with zero interpreter overhead.

Paper 14: POET — Training Neural Networks on Tiny Devices with Integrated Rematerialization and Paging (ICML 2022)

- **Authors:** Shishir G. Patil, Paras Jain, Prabal Dutta, Ion Stoica, Joseph E. Gonzalez (UC Berkeley)
- **Conference:** ICML 2022
- **Direct Link:** https://arxiv.org/abs/2207.07697
- **Direct PDF:** https://arxiv.org/pdf/2207.07697.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Even with small models, during training there is often a "peak memory spike" where several large feature tensors must exist at the exact same moment. If that spike exceeds the microcontroller's SRAM by even 1 byte, the microcontroller crashes with a HardFault or Out-of-Memory error.
- **The Breakthrough Idea:** POET introduces **Integrated Rematerialization and Paging**:
  1. **Rematerialization (Recomputation):** Why store an activation in SRAM if it only takes a few microseconds to recompute? POET deletes cheap activations and simply re-runs the forward calculation when the backward pass needs them!
  2. **Paging:** For activations that are too slow to recompute, it temporarily moves them to secondary storage (like Flash or SD card) and pages them back when needed.
  3. It frames edge memory management as an optimization algorithm to find the lowest energy schedule that fits within a fixed memory budget.

---

2. Step-by-Step Workflow

1. **Profile Graph:** Measure the exact computation time and memory size of every single forward and backward operation.
2. **Cost-Benefit Analysis:** For every intermediate activation tensor, compare:
   - Cost of keeping it in SRAM (occupies memory).
   - Cost of recomputing it (costs a few CPU clock cycles).
   - Cost of paging it to Flash (costs Flash write energy).
3. **Formulate Integer Linear Program (ILP):** An optimization algorithm computes the best schedule.
4. **Execution:** Cheap activations are discarded and recomputed; expensive activations are kept or paged.
5. **Result:** Trains models on devices with **2× to 4× less RAM** with minimal energy penalty.

---

3. Diagram to Draw in Your Notebook

```
STANDARD TRAINING (Stores everything -> OOM Crash):
Forward:  [Act 1] ──> [Act 2] ──> [Act 3] ──> Loss
SRAM:     |========= Full Storage in RAM =========|  --> CRASH (>64KB)

POET REMATERIALIZATION (Discards & Recomputes):
Forward:  [Act 1] ──> [Act 2] ──> [Act 3] ──> Loss
SRAM:     | Only Act 1 is saved in RAM |  (Act 2 & 3 DELETED!)

Backward: Loss ──> [Recompute Act 3] ──> Backprop Act 3
               ──> [Recompute Act 2] ──> Backprop Act 2
               ──> Use saved Act 1   ──> Backprop Act 1
SRAM:     | Memory stays flat and safe under 64KB! |
```

---

4. Key Formulas & Methodology

1. Memory Budget Constraint:

$$\sum_{i \in \text{Live}(t)} \text{Size}(T_i) \leq M_{\text{SRAM}}$$

- *Explanation:* At every single time step t, the sum of memory occupied by all active tensors must never exceed the target SRAM budget (e.g., 64KB).

2. Rematerialization Trade-Off:

$$T_{\text{total}} = T_{\text{forward}} + T_{\text{backward}} + T_{\text{recompute}}$$

- *Explanation:* You spend roughly 20%–30% more CPU time recomputing cheap layers, but in return, the model fits into a microcontroller that would otherwise be completely unable to run it.

---

5. Viva / Exam Questions for Devrat

- **Q: What is "Rematerialization"?**
  Answer: Rematerialization means deleting intermediate forward activations to save SRAM, and recalculating them on-the-fly when they are needed during the backward pass.
- **Q: Why is rematerialization useful for microcontrollers?**
  Answer: Because microcontrollers have fast CPUs (ESP32 runs at 240 MHz) but severely limited SRAM (only 520KB). Trading cheap CPU clock cycles to save precious SRAM is the smartest trade-off.

Paper 15: Training Deep Nets with Sublinear Memory Cost (Gradient Checkpointing)

- **Authors:** Tianqi Chen, Bing Xu, Chiyuan Zhang, Carlos Guestrin
- **Publication:** arXiv preprint (arXiv:1604.06174), 2016
- **Direct Link:** https://arxiv.org/abs/1604.06174

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** In a neural network with N layers, backpropagation normally requires storing all N feature maps in RAM. The memory complexity is $O(N)$. For very deep networks (e.g., 100 layers), memory scales linearly, blowing up the hardware memory limit.
- **The Breakthrough Idea:** This seminal paper invented **Gradient Checkpointing**:
  - Instead of saving all N activations, save only a small subset of $\sqrt{N}$ activations (called **checkpoints**).
  - During backpropagation, recompute the missing intermediate activations inside each segment starting from the nearest checkpoint.

- This reduces the memory complexity from **linear $O(N)$ to sublinear $O(\sqrt{N})$**, while adding only **one extra forward pass (~30% compute overhead)**!

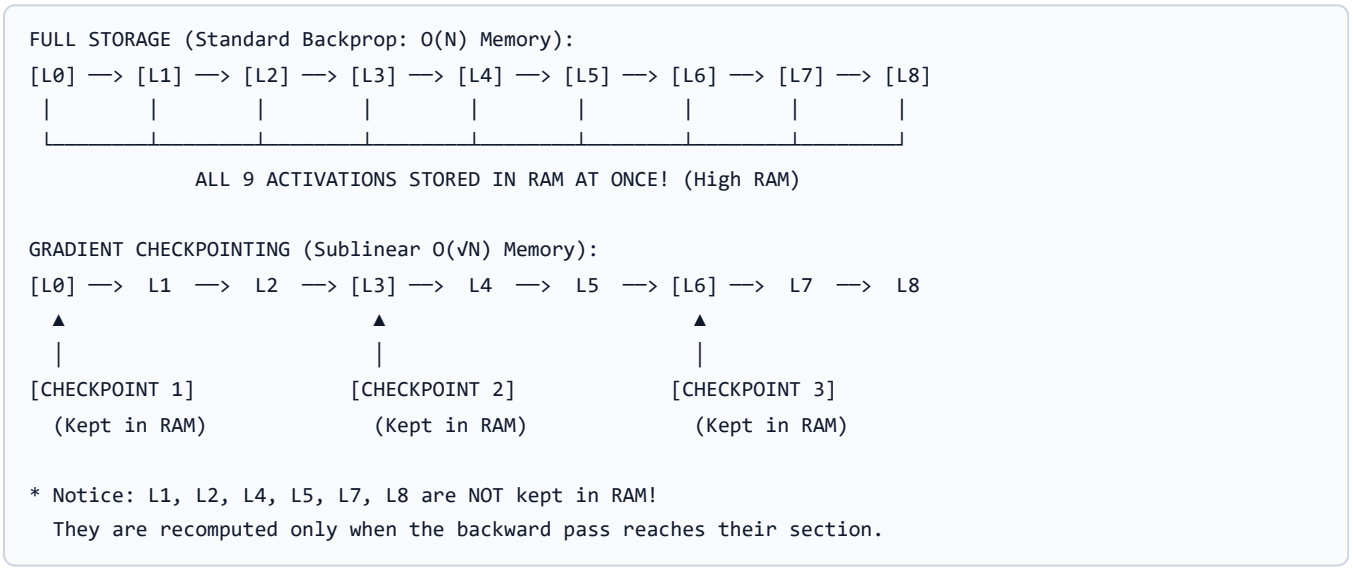
---

2. Step-by-Step Workflow

1. **Divide Network into Segments:** For an N-layer network, split the layers into $\sqrt{N}$ blocks, each of size $\sqrt{N}$.
2. **Forward Pass:** Run the forward pass normally, but **only keep the output of each segment boundary** in memory (these are the Checkpoint Nodes). Discard all internal nodes inside each segment.
3. **Reach Loss:** Compute the final loss node.
4. **Backward Pass Segment-by-Segment:**
   - Take the checkpoint at the start of the last segment.
   - Recompute the missing internal activations of that segment.
   - Run the backward pass through that segment.
   - Free the segment's memory and move to the previous segment.
5. **Memory Stays Bounded:** Peak memory never exceeds $O(\sqrt{N})$.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Mathematical Proof of Optimal Segment Size:

- Let N be total layers, and let checkpoints be placed every k layers.
- Total memory cost function:

$$M(k) = \frac{N}{k} + k$$

- To find the minimum memory, take the derivative with respect to k and set it to 0:

$$\frac{dM}{dk} = -\frac{N}{k^2} + 1 = 0 \implies k^2 = N \implies k = \sqrt{N}$$

- Therefore, the optimal checkpoint distance is $k = \sqrt{N}$, yielding $O(\sqrt{N})$ peak memory!

2. Computational Overhead:

$$T_{\text{extra}} \leq 1 \times T_{\text{forward}}$$

- *Explanation:* Gradient checkpointing requires at most one extra forward pass (~30% runtime overhead). Saving 80% RAM for just 30% more computation is a massive win on embedded hardware.

---

5. Viva / Exam Questions for Devrat

- **Q: What is the asymptotic memory complexity of standard backpropagation vs gradient checkpointing?**
  Answer: Standard backpropagation is linear $O(N)$. Gradient checkpointing is sublinear $O(\sqrt{N})$, where N is the number of layers.
- **Q: How does this apply to MTC?**
  Answer: In our compiler's Phase 3 (Static Memory Planner), when models grow deeper than 3 or 4 layers, MTC can insert checkpointing boundaries into the `IrGraph`, ensuring the peak SRAM buffer never exceeds the 64KB ESP32 watermark.

Quick Reference Summary Table for Devrat

| Paper | Main Breakthrough | Memory Saved | Core Mechanism | How MTC Uses It | | :--- | :--- | :--- | :--- | :--- | | **MCUNetV3** (NeurIPS 2022) | Full on-device training under 256KB | > 90% RAM | Sparse layer/tensor updates + AOT code generation | Confirms MTC's AOT static compilation design | | **TinyTL** (NeurIPS 2020) | Reduce activation memory, not parameters | 6.5× RAM | Freeze backbone; update only biases + LiteResidual | Mathematical proof that activations dominate RAM | | **MCUNet** (NeurIPS 2020) | Co-design: TinyNAS + TinyEngine | 3× RAM | Eliminates runtime interpreter; in-place buffer reuse | Direct inspiration for MTC's zero-dependency C99 emission | | **POET** (ICML 2022) | Memory-bounded training on tiny devices | 2× to 4× RAM | Rematerialization (recomputing cheap layers) + Paging | Strategy to keep peak SRAM strictly under 64KB | | **Sublinear Memory** (2016) | Gradient Checkpointing | O(N) → O(√N) | Save √N checkpoints; recompute intermediate steps | Extends MTC to deep multi-layer neural architectures |